

Низькорівневе програмування та програмування мікроконтролерів

З використанням мови програмування **Assembler**
Робота з пам'яттю

класрум **im3ozqq4**

Питання

Звісно, реєстри це добре,
Але що робити коли код є достатньо складний,
реєстрами не обмежитися, да і ми звикли до
значень змінних **x, y, z...**

Є рішення

Ми вже знаємо про "мітки", які грають роль адресів

```
mov dx, x
...
...
...
...
x:
```

Але їх не можна
використовувати для даних

Бо записуючи дані по адресі x ,
де знаходиться код, ми
“перетремо” код програми

Розділення коду і даних

Це одна з основних проблем у усіх мовах програмування без виключення

Синтаксис

Ми завжди явно
Розділюємо код і дані

```
.text
_start:
    xor al, al
    ...

.data
x: .byte 1
```

Розмір має значення

В залежності від використаного синтаксису буде виділена різна кількість байтів

```
.data  
a:  .byte 1  
b:  .word 65536  
c:  .long 2000000000  
d:  .quad 0x0123456789abcdef
```


Але треба бути дуже уважним

```
mov al, [a]
```

```
.data
```

```
a: .byte 0x01
```

```
b: .word 0x2345
```

```
c: .long 0x6789abcd
```

```
d: .quad 0x0123456789abcdef
```

Але треба бути дуже уважним

```
mov ax, [a]
```

```
.data
```

```
a: .byte 0x01
```

```
b: .word 0x2345
```

```
c: .long 0x6789abcd
```

```
d: .quad 0x0123456789abcdef
```


Але треба бути дуже уважним

```
mov eax, [a]
```

```
.data
```

```
a: .byte 0x01
```

```
b: .word 0x2345
```

```
c: .long 0x6789abcd
```

```
d: .quad 0x0123456789abcdef
```

Але треба бути дуже уважним

```
mov rax, [a]

.data
a:  .byte 0x01
b:  .word 0x2345
c:  .long 0x6789abcd
d:  .quad 0x0123456789abcdef
```


Виділення місця під рядок

```
.data  
text: .ascii "hello"
```

Виділення місця під рядок

дуже часто потрібен рядок, який закінчується на 0

```
.data  
text1: .ascii "hello"  
| | | | .byte 0  
text2: .asciz "hello"
```

В другому випадку 0 буде додано автоматично

Дуже часто потрібна довжина рядка

```
.data  
text: .ascii "hello"  
len:   .byte 5
```

Дуже часто потрібна довжина рядка

```
.data  
text: .ascii "hello"  
len:   .byte 5
```

Але легко помилитися

Є спеціальний синтаксис якій рахує відстань

```
.data  
text: .ascii "hello"  
len  = . - text
```

Довжина буде порахована автоматично

Є спеціальний синтаксис якій рахує відстань

```
.data  
text: .ascii "hello"  
len  = . - text
```

Довжина буде порахована автоматично
Місце зайнято не буде


```
mov al, len  
mov ax, len  
mov eax, len  
  
.data  
text: .ascii "hello"  
len = . - text
```

Відпрацює правильно, вне залежності від довжини регістру

Деякі нюанси синтаксису різних діалектів асемблеру

```
.data  
a:  .byte 0x01  
b:  .word 0x2345  
c:  .long 0x6789abcd  
d:  .quad 0x0123456789abcdef
```

```
segment readable writeable  
a db 0x01  
b dw 0x2345  
c dd 0x6789abcd  
d dq 0x0123456789abcdef
```

одне й те саме

Виділення однакових даних

Альтернативний синтаксис

```
a1 db 0x01, 0x01, 0x01, 0x01
```

```
a2 db 4 dup(0x01)
```

Виділення неініціалізованої області пам'яті

```
buffer db 256 dup(?)
```


Масиви?

Масиви?

...руками

Масиви

```
aa db 1,2,3,4,5,6,7,8  
size1 equ $ - aa
```

```
ab db 8 dup (?)  
size2 equ $ - aa
```

Вирівнювання

01	00	00	00	00	00	00	00	d5	ff	ff	ff	ff	4f	00	00
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
09	00	00	00	00	00	00	00	00	10	40	00	00	00	00	00
05	00	00	00	00	00	00	00	05	00	00	00	00	00	00	00
04	00	00	00	00	00	00	00	38	00	00	00	00	00	00	00
03	00	00	00	00	00	00	00	40	00	40	00	00	00	00	00
1f	00	00	00	00	00	00	00	de	ff	ff	ff	ff	4f	00	00
19	00	00	00	00	00	00	00	e7	ff	ff	ff	ff	4f	00	00

Чому не працює

```
mov ax, [dx]
```

Тому використовуємо

```
mov ax, [edx]
```


Notes

Notes

Notes

Notes

Notes

Notes

Notes

Notes

Notes